

Microservices Interview Guide

29 questions — Easy → Medium → Hard

One-page map (revision table)

Tier	Topics (short)	Interview expectation
Easy	What MS is, monolith vs MS, pros/cons, gateway, discovery, load balancing, REST, config, logging	Clear definitions + 1 example each
Medium	Circuit breaker, retry, tracing, Spring Cloud stack, Eureka, Feign, Gateway/Config, sync/async, event-driven	Patterns + when to use + pitfalls
Hard	Saga, idempotency, consistency/outbox, API versioning, bulkhead/rate limit, OAuth2/JWT, fault tolerance, Docker/K8s, end-to-end design	Trade-offs + failure scenarios + design walkthrough

Part A — Easy (foundation)

These should be **automatic** in an interview: correct, crisp, and confident.

Q1. What are microservices?

In one line: An architecture where the system is split into **small, independently deployable services**, each owning a **business capability**, talking over the network (APIs / messages).

Say this in the interview

- Each service runs in **its own process** (often its own container).
- **Loose coupling** — change one service without redeploying everything.
- **Independent scale** — scale only the hot path (e.g. checkout), not the whole app.

Contrast (one sentence): A **monolith** is one deployable unit; **microservices** are many deployables coordinated by contracts (APIs, events).

Q2. Monolith vs microservices — key differences

Dimension	Monolith	Microservices
Deploy unit	Single artifact	Many services
Scaling	Scale whole app	Scale per service
Failure blast radius	Often larger	Can be smaller <i>if</i> designed well
Dev speed (early)	Usually faster	Slower until platform maturity

Ops complexity	Lower initially	Higher (many moving parts)
----------------	-----------------	----------------------------

Interview tip: Always add “**it depends on team size and maturity**” — interviewers like pragmatism.

Q3. Advantages of microservices

- **Independent deployment** — faster release cadence per domain team.
 - **Technology flexibility** — different services, different stacks (where justified).
 - **Fault isolation** — one bad service shouldn't take down everything (with resilience patterns).
 - **Parallel teams** — clearer ownership per bounded context.
-

Q4. Disadvantages / costs of microservices

- **Operational load** — many services, pipelines, clusters, observability.
- **Distributed systems problems** — latency, partial failure, **eventual consistency**.
- **Network overhead** — serialization + HTTP/gRPC vs in-process calls.
- **Testing & debugging** — need contract tests, integration environments, **distributed tracing**.

One line: Microservices buy **scale and autonomy**; you pay in **complexity and discipline**.

Q5. What is an API Gateway?

Role: **Single entry** for clients (web, mobile, BFF).

Typical responsibilities

- Authentication / TLS termination

- **Routing** to the right service
- Rate limiting, caching, request shaping
- Hiding internal topology from clients

Examples: Spring Cloud Gateway, Kong, AWS API Gateway.

Q6. What is service discovery?

Problem: IPs and instance counts change (especially on Kubernetes).

Idea: Services **register**; consumers **look up** instances from a **registry** (or via DNS / mesh).

Style	Idea	Example vibe
Client-side	Client asks registry, picks instance	Eureka + client LB
Server-side	LB / ingress resolves instances	AWS ELB/ALB, K8s Service

Q7. Load balancing in microservices

Goal: Spread traffic across **healthy** instances.

- **Client-side:** Client (or sidecar) picks instance from list (e.g. Spring Cloud LoadBalancer).
- **Server-side:** Nginx, cloud LB, K8s Service → kube-proxy.

Algorithms (name-drop): round-robin, least connections, weighted, health-aware.

Q8. REST between services

- HTTP verbs, **stateless** requests, JSON/XML payloads.
 - **Pros:** simple, debuggable, universal tooling.
 - **Cons: temporal coupling** — caller waits; failures and retries need design (idempotency).
-

Q9. Configuration management

Goal: No secrets or URLs **hardcoded** in the binary; **environment-specific** config externalized.

- Central store (Git, Vault, parameter store).
- Services fetch at startup; optional **runtime refresh** (bus + `@RefreshScope` in Spring world).

Interview phrase: “**12-factor** style config — same image, different config per env.”

Q10. Centralized logging

Problem: Logs scattered across pods/services.

Approach: Ship logs to one stack — **ELK** (Elasticsearch, Logstash, Kibana) or **EFK** (Fluentd instead of Logstash).

Must-have: **Correlation / trace ID** on every log line so one user request can be stitched across services.

Part B — Medium (real interview depth)

Expect “**why?**”, “**what breaks?**”, “**what would you use at work?**”

Q11. Circuit breaker pattern

Problem: Slow/failing downstream → threads pile up → **cascading failure**.

States

State	Behaviour
Closed	Normal traffic
Open	Fail fast — don't call broken dependency
Half-open	Trial calls to test recovery

Java ecosystem: Resilience4j (preferred today), Hystrix (legacy knowledge only).

Say this: “Breaker pairs with **timeouts** and **bulkheads**; alone it's not enough.”

Q12. Retry mechanism

Use for: Transient failures (blips, 503, timeouts).

Design knobs

- Max attempts, backoff, **jitter** (avoid thundering herd)
- **Don't retry blindly** on non-idempotent operations without keys

Critical pairing: Retry + **circuit breaker** — otherwise you hammer a dying service.

Libs: Resilience4j, Spring Retry.

Q13. Distributed tracing

Concepts

- **Trace ID** — one logical request across the system
- **Span** — one unit of work inside a service

Tools: Zipkin, Jaeger; Spring → **Micrometer Tracing** (Sleuth evolved into this story).

Interview win: “Tracing turns ‘which microservice?’ from hours into minutes.”

Q14. Role of Spring Cloud (Java microservices)

Spring Cloud bundles **patterns** on Spring Boot:

Concern	Typical Spring Cloud piece
Config	Spring Cloud Config
Discovery	Eureka (or K8s-native alternatives)
Gateway	Spring Cloud Gateway
HTTP clients	OpenFeign
Resilience	Resilience4j integration

Tracing	Micrometer / tracing bridges
---------	------------------------------

Framing: “It’s not magic — it’s **opinionated glue** for common distributed patterns.”

Q15. Netflix Eureka

- **Service registry** — register on startup, **heartbeats** for health.
 - Clients discover **instance lists** instead of hardcoded hosts.
 - **HA** via peer replication; tolerates certain partition behaviours (know Eureka vs strong CP systems at a high level).
-

Q16. OpenFeign (Feign client)

- **Declarative** HTTP client — interface + annotations → runtime proxy.
- Integrates with discovery + resilience (timeouts, CB, retries when configured).

vs RestTemplate: Less boilerplate; clearer boundaries between services.

Q17. Zuul vs Spring Cloud Gateway

- **Zuul:** Netflix-era; older story in greenfield Spring.
- **Spring Cloud Gateway:** reactive (WebFlux/Reactor), predicates + filters, modern default for **edge routing** in Spring stacks.

You should explain: predicates (path, header), filter chain, where auth runs.

Q18. Spring Cloud Config Server

- Config in **Git** (or Vault/S3 patterns).
- Services pull via HTTP; **refresh** via Bus (Kafka/Rabbit) for selective reload.
- Supports **environment-specific** labels/profiles.

Q19. Inter-service communication — sync vs async

	Synchronous (REST, gRPC)	Asynchronous (Kafka, RabbitMQ, SQS)
Coupling	Temporal — caller waits	Time decoupling
Consistency reasoning	Often “easier” mentally	Eventual consistency
Failure	Propagates unless contained	Needs idempotency, dedup, DLQ
Good for	Query, immediate read-after-write feel	Notifications, workflows, fan-out

Rule of thumb: Prefer **async** when you don’t need the answer in the same request thread; prefer **sync** when UX needs a strong immediate response.

Q20. Event-driven architecture (EDA)

- Producers emit **events**; consumers react **without tight knowledge** of each other.
- Broker: Kafka / RabbitMQ / cloud equivalents.

- Enables **scale**, **decoupling**, **audit trails** — at the cost of harder debugging and **schema evolution** discipline.

Buzzwords to use correctly: topics/partitions (Kafka), exchanges/queues (Rabbit), **consumer groups**, **dead-letter queues**.

Part C — Hard (where answers separate levels)

Focus on **patterns + failure + mitigation + trade-offs**.

Q21. Distributed transactions — Saga pattern

Reality: No single ACID transaction across independent databases.

Saga: Sequence of **local transactions**; each step publishes the next trigger (event/command).

Style	Idea	Trade-off
Choreography	Services react to events	Can become hard to “see” the full flow
Orchestration	Central coordinator drives steps	Coordinator is a critical component

Failure: **Compensating transactions** roll back logical work (e.g. cancel reservation, refund).

Q22. Idempotency

Definition: Doing an operation **twice** is safe — same outcome as once.

Why it matters: Retries, duplicate broker delivery, client timeouts → **double charges**, double inventory, etc.

Implementation sketch

- Client sends **idempotency key**
 - Server stores (**key** → **result**) or uses natural unique constraints
 - Replays return stored outcome
-

Q23. Data consistency across services

- **Database per service** — no shared DB anti-pattern (in principle).
 - **Eventual consistency** is normal — explain that calmly.
 - **Saga** for multi-step business processes.
 - **Outbox pattern** — write business row + **outbox event** in **same DB transaction**; publisher drains outbox → broker (no “dual write” bug).
 - **Event sourcing** (where appropriate) — immutable event log as source of truth.
-

Q24. API versioning

Strategies

- **URI:** `/api/v1/...` — obvious, cache-friendly caveats
- **Header / Accept** — cleaner URLs, more discipline needed

Good citizenship: deprecation policy, changelog, backward compatible changes when possible, **semantic versioning** mindset.

Q25. Bulkhead & rate limiting

Bulkhead: Isolate pools (threads, connections) per dependency — **one slow service can't starve the JVM.**

Rate limiting: Protect from abuse / overload — often at **gateway** (e.g. Redis-backed limiter in Spring Cloud Gateway) or **mesh** (Istio).

Q26. Security — JWT & OAuth2 / OIDC

Typical flow

1. **Authorization Server** authenticates user/client
2. Issues **access token** (often **JWT**)
3. **API Gateway** validates signature, expiry, **scopes/roles**
4. Downstream services trust gateway or re-validate

Must mention: HTTPS everywhere, **short-lived access tokens**, refresh tokens, **no secrets in JWT** (claims aren't encryption).

Q27. Failures & fault tolerance (layered)

Application

- Timeouts
- **Circuit breaker**
- **Retry** with backoff + jitter
- **Bulkhead**
- **Fallback** (degraded mode — be honest about staleness)

Platform

- Health checks (readiness vs liveness)
 - **K8s** restarts, multiple replicas
 - **Stateless** services behind LB for fast failover
-

Q28. Docker & Kubernetes

Docker: Immutable, portable **unit of deployment** — deps + app bundled.

Kubernetes: Scheduling, **rolling updates**, **HPA**, self-heal, service discovery via **DNS/Services**, namespaces for isolation, **Helm** for packaging.

One sentence: “Docker packages; Kubernetes **operates** at fleet scale.”

Q29. End-to-end scalable microservices design (sample Spring-flavoured answer)

Narrative you can walk on a whiteboard

1. **DDD** → bounded contexts → service boundaries
2. **DB per service**
3. **API Gateway** — auth, routing, rate limits
4. **Discovery** (Eureka or K8s-native)
5. **Sync** Feign/gRPC for request/response paths; **async** Kafka for domain events
6. **Config Server** + secrets management
7. **Resilience** — CB, retry, timeout, bulkhead
8. **Observability** — metrics (Prometheus), dashboards (Grafana), logs (ELK/EFK), traces (Zipkin/Jaeger)
9. **CI/CD**, **contract tests**, **feature flags** where relevant

Important: Name-dropping tools is useless without **why each piece exists**.

Study checklist (honest self-audit)

Easy — can I explain without slides?

- Monolith vs MS with one real-world analogy
- Gateway vs service vs client responsibilities
- Why discovery exists in cloud/K8s
- Why centralized logs need trace IDs

Medium — can I sketch a diagram?

- Request with gateway + 3 services + DB-per-service
- Circuit breaker timeline (closed → open → half-open)
- One sync flow and one async flow for the same business (e.g. order placed)

Hard — can I defend trade-offs?

- Saga choreography vs orchestration with one failure example
- Outbox vs naive “write DB then publish to Kafka”
- Idempotency for payment / inventory
- JWT validation at gateway vs service

Final interview tips (India IT context)

1. **Start with business context** — “We split order and payment because they scale and change independently...”
2. **Always mention ops** — logging, metrics, tracing — it signals production maturity.
3. **Never claim ‘strong ACID everywhere’** across services — show you understand **eventual consistency**.
4. **Spring-heavy panels** — tie patterns to **Spring Cloud Gateway, OpenFeign, Resilience4j, Config, Micrometer** tracing.
5. **Cloud-native panels** — be ready to pivot vocabulary from Eureka to **Kubernetes Services, Ingress, service mesh**.

